

Tema 3 - Funciones

Contenido Teórico

Objetivos

- **Entender una función** como un bloque de código reutilizable que encapsula una tarea concreta.
- **Aprender la sintaxis** `def`, la llamada a una función, los parámetros, los argumentos y el valor devuelto con `return`.
- **Diferenciar claramente** lo que es imprimir información por pantalla de lo que es devolver resultados reutilizables por otras partes del programa.
- **Usar argumentos** posicionales, argumentos con nombre, valores por defecto y un número variable de argumentos.
- **Aplicar buenas prácticas básicas:** nombres claros, funciones pequeñas, docstrings útiles y separación entre cálculo y presentación.

Por qué usar funciones

Una **función** permite dar nombre a una operación y reutilizarla sin copiar el mismo bloque varias veces.

- Esto reduce errores, mejora la lectura y facilita el mantenimiento.
- En scripts de soporte es habitual repetir tareas como normalizar texto, calcular importes, validar estados o construir mensajes. Si esas tareas se encapsulan en funciones, el programa queda más ordenado.
- Una buena función debe tener un propósito claro. Si hace demasiadas cosas, probablemente conviene dividirla.

```
def estado_cpu(uso):  
    if uso >= 90:  
        return "CRÍTICO"  
    if uso >= 70:  
        return "ALTO"  
    return "NORMAL"  
  
print(estado_cpu(82))
```

Sintaxis básica con def

Sintaxis:

```
def nombre_función():
```

- El bloque de instrucciones de la función se delimita con indentación.
- Definir una función no la ejecuta. Sólo queda disponible para que el programa pueda llamarla más adelante.

La ejecución ocurre cuando se escribe su nombre seguido de paréntesis:

```
nombre_función()
```

- El nombre debe ser descriptivo y normalmente se escribe en estilo **snake_case**, igual que las variables.

```
def saludar_operador():  
    mensaje = "Sesión de soporte iniciada"  
    print(mensaje)  
  
saludar_operador()
```

Llamada a una función

Llamar a una función significa ejecutar el bloque que se definió previamente. La llamada puede hacerse una vez, muchas veces o dentro de otras estructuras como condicionales y bucles.

- **Cada llamada es independiente:** Python entra en la función, ejecuta su código y vuelve al punto desde el que fue invocada.
- Este mecanismo permite construir programas por piezas pequeñas y comprobables.

```
def mostrar_estado(servicio):  
    print(f"Comprobando servicio: {servicio}")  
  
mostrar_estado("ssh")  
mostrar_estado("httpd")  
mostrar_estado("backup")
```

Parámetros y Argumentos

Los parámetros son los nombres definidos en la cabecera de la función. Los argumentos son los valores concretos que se pasan cuando la función se llama.

- Esta diferencia ayuda a leer documentación y a diagnosticar errores.
- Si una función espera dos parámetros y la llamada solo envía uno, Python no podrá ejecutarla correctamente.
- Los parámetros actúan como variables locales dentro de la función.

```
def calcular_iva(base, porcentaje):  
    impuesto = base * porcentaje / 100  
    return impuesto  
  
print(calcular_iva(100, 21))  
print(calcular_iva(250, 10))
```

return frente a print

- `print()` muestra información por pantalla, pero no entrega un resultado para que el programa lo siga usando.
- `return` finaliza la función y devuelve un valor al código que la llamó.
- En código profesional conviene que las funciones calculen y devuelvan resultados.
- La presentación por pantalla puede hacerse fuera. Esto permite reutilizar la función en scripts, notebooks, módulos o pruebas.
- Una función puede imprimir algo, pero imprimir no sustituye a devolver.

```
def calcular_descuento(precio, descuento):  
    return precio - (precio * descuento / 100)
```

```
precio_final = calcular_descuento(80, 15)
print(f"Precio final: {precio_final:.2f} €")
```

Funciones sin return explícito

Si una función no ejecuta una instrucción `return`, Python devuelve automáticamente `None`.

- Esto no es un error, pero hay que entenderlo para no intentar operar con un resultado inexistente.
- Las funciones que solo imprimen mensajes pueden ser útiles para informar, pero no sirven para encadenar cálculos.
- Cuando el resultado de una función tenga que usarse después, debe devolverse explícitamente con `return`.

```
def registrar_evento(texto):
    print(f"EVENTO: {texto}")

resultado = registrar_evento("reinicio programado")
print(resultado)  # Devuelve None
```

Devolver varios valores

En Python una función puede devolver varios valores separados por comas.

- En realidad, Python agrupa esos valores en una tupla y permite recuperarlos con varias variables.
- Esta técnica es práctica cuando una operación produce varios resultados relacionados, por ejemplo, un mínimo, un máximo y una media.
- **Debe usarse con moderación:** si se devuelven demasiados datos, puede ser señal de que la función necesita rediseñarse.

```
def resumen_tiempos(valores):
    minimo = min(valores)
    maximo = max(valores)
    media = sum(valores) / len(valores)
    return minimo, maximo, media

mn, mx, avg = resumen_tiempos([12, 9, 15, 11])
print(mn, mx, round(avg, 2))
```

Alcance de variables

- **Las variables creadas dentro de una función tienen alcance local:** existen durante la ejecución de esa función y no deben usarse directamente desde fuera.
- Una función puede leer argumentos recibidos y crear variables internas para resolver su tarea.
- El resultado debe salir mediante `return`.
- Esta separación evita efectos inesperados y hace que el código sea más fácil de probar y mantener.

```
def construir_ruta(base, fichero):
    ruta = f"{base}/{fichero}"
    return ruta

ruta_log = construir_ruta("/var/log", "app.log")
```

```
print(ruta_log)
# print(ruta) # fuera de la función no existe
```

Argumentos posicionales

Los argumentos posicionales (sin indicar nombre) se asignan a los parámetros según el orden en que aparecen.

- Son sencillos, pero pueden provocar errores si el orden no es evidente.
- Funcionan bien cuando hay pocos argumentos y su significado es claro.
- Si una función recibe varios datos del mismo tipo, puede ser más seguro llamar usando nombres.
- En funciones críticas, la claridad de la llamada es tan importante como la lógica interna.

```
def crear_usuario(nombre, grupo, activo):
    return f"{nombre} -> {grupo} -> activo={activo}"

print(crear_usuario("ana", "operadores", True))
# Esta segunda invocación devuelve mal ordenado
print(crear_usuario("operadores", "ana", True))
```

Argumentos con nombre

Los argumentos con nombre permiten indicar explícitamente qué valor corresponde a cada parámetro.

- Mejoran la legibilidad y permiten cambiar el orden de la llamada.
- Son especialmente útiles cuando una función tiene varios parámetros opcionales o cuando dos parámetros son del mismo tipo y podrían confundirse.
- En scripts de administración ayudan a que el código se lea casi como una configuración.

```
def programar_tarea(nombre, hora, habilitada):
    return f"{nombre} a las {hora}, habilitada={habilitada}"

print(programar_tarea(nombre="backup", hora="02:00", habilitada=True))
print(programar_tarea(habilitada=False, hora="03:00", nombre="limpieza"))
```

Valores por defecto

Un parámetro puede tener un valor por defecto.

- Si la llamada no proporciona ese argumento, Python usa el valor definido en la cabecera.
- Los valores por defecto permiten simplificar llamadas habituales sin perder flexibilidad para casos concretos.
- Por convención, los parámetros obligatorios van antes y los parámetros con valor por defecto se colocan después.
- También se puede indicar el tipo de retorno mediante anotaciones, por ejemplo `-> str` o `-> float` | `None` cuando el resultado pueda ser `None`.

```
def crear_alerta(mensaje, nivel="INFO", notificar=False) -> str:
    texto = f"[{nivel}] {mensaje}"
    if notificar:
        texto += " -> enviar aviso"
    return texto

print(crear_alerta("Servicio iniciado"))
print(crear_alerta("Disco lleno", nivel="CRITICO", notificar=True))
```

Cuidado con valores mutables por defecto

No conviene usar listas, diccionarios u otros objetos mutables como valor por defecto. Ese objeto se crea una sola vez al definir la función y puede conservar cambios entre llamadas.

- El caso típico es una función que genera un historial. Si se usa `[]` como valor por defecto, este lo toma el parámetro como valor al crear la función. Llamadas posteriores pueden reutilizar la misma lista sin que sea evidente.

En el ejemplo, si usamos la forma mal construida, lo que se observa es que los valores que retorna son impredecibles. Si le damos el argumento `historico_eventos`, agrega el valor a esa lista, pero si no pasamos el segundo argumento, historial va acumulando los valores progresivamente y son los que retorna al invocarlo

La práctica segura es usar `None` como valor por defecto y crear la lista dentro de la función cuando no se haya recibido una lista externa. Eso la inicializa en cada ejecución y parte de cero

```
# Mal construido. El argumento se crea al inicio
def agregar_evento(evento, historial=[]):
    historial.append(evento)
    return historial

historico_eventos = []
print(agregar_evento("arranque", historico_eventos))
print(agregar_evento("parada"))

# Bien construido. El argumento se inicializa en cada ejecución
def agregar_evento(evento, historial=None):
    if historial is None:
        historial = []
    historial.append(evento)
    return historial

historico_eventos = []
print(agregar_evento("arranque", historico_eventos))
print(agregar_evento("parada"))
```

Número variable de argumentos: `*args`

- `*args` permite que una función reciba un número variable de argumentos posicionales. Dentro de la función se reciben como una tupla (las veremos más adelante).
 - Es útil cuando la cantidad de valores no es fija, por ejemplo, al calcular un total, revisar varias rutas o construir un mensaje a partir de varias partes.
 - No debe usarse para ocultar una interfaz confusa. Si la función necesita datos concretos, es mejor declararlos como parámetros normales.

```
def totalizar_consumo(*medidas):
    total = 0
    for valor in medidas:
        total += valor
    return total

print(totalizar_consumo(12, 15, 9))
print(totalizar_consumo())
```

Argumentos por nombre variables: **kwargs

- ****kwargs** permite recibir un número variable de argumentos con nombre.
 - Dentro de la función se reciben como un diccionario.
 - Es útil para metadatos, opciones de configuración o atributos variables.
 - No sustituye a definir parámetros claros cuando la función necesita información obligatoria.
 - En este tema basta con reconocerlo y usarlo de forma básica. Los diccionarios se tratarán con más detalle en el tema de colecciones.

```
def describir_recurso(nombre, **atributos):
    print(f"Recurso: {nombre}")
    for clave, valor in atributos.items():
        print(f"- {clave}: {valor}")

describir_recurso("srv01", cpu=4, memoria="16GB", entorno="prod")
```

Orden recomendado de parámetros

Cuando una función combina diferentes tipos de parámetros, el orden importa.

- Primero se colocan los parámetros obligatorios, después ***args**, luego parámetros con nombre opcionales y finalmente ****kwargs**.
- No es necesario usar todas estas posibilidades en cada función. De hecho, una función sencilla suele ser preferible.
- La firma de la función debe ayudar a entender cómo se llama y qué datos necesita.

```
def registrar_metricas(servicio, *valores, unidad="ms", **etiquetas):
    print(f"Servicio: {servicio}")
    print(f"Valores: {valores} {unidad}")
    print(f"Etiquetas: {etiquetas}")

registrar_metricas("api", 12, 15, 9, unidad="ms", zona="dmz")
```

Anotaciones de tipo y docstrings

Las anotaciones de tipo permiten indicar qué tipos se esperan y qué tipo se devuelve. Python no las fuerza automáticamente, pero ayudan a documentar y a que el editor detecte problemas.

- Cada parámetro indica a la derecha con dos puntos el tipo. Por ejemplo, `valor: float` o `nombre: str`. El valor de retorno se indica al cierre de paréntesis con flecha. Ejemplo `-> float: 0 -> str:`

- El **docstring** al principio de la función describe la intención de esta, sus argumentos y el resultado. Es una buena práctica. No debe ser una repetición mecánica del código.
- Si invocamos la función con `función.__doc__` muestra el **docstring** como ayuda.
- Juntas, anotaciones y docstrings hacen que la función sea más fácil de usar por otras personas.

```
def calcular_porcentaje(valor: float, total: float) -> float:
    """Devuelve el porcentaje que representa valor sobre total."""
    return valor * 100 / total

print(calcular_porcentaje(45, 200))
print(calcular_porcentaje.__doc__)
```

Buenas prácticas y errores frecuentes

Buenas Prácticas:

- Una función debe tener un nombre claro, hacer una tarea reconocible y devolver un resultado cuando ese resultado deba reutilizarse.
- Evitar funciones demasiado largas, nombres genéricos como "proceso" o "datos", y mezclas innecesarias de cálculo, entrada de usuario y presentación.

Errores frecuentes:

- Olvidar llamar a la función
- Confundir print con return
- Pasar argumentos en orden incorrecto
- No contemplar None
- Crear variables locales esperando usarlas fuera.

```
def calcular_iva(precio: float) -> float:
    """Calcula el 21% de IVA de un precio."""
    return precio * 0.21

impuesto = calcular_iva(100.0)
```

Resumen del Tema 3

- Las funciones permiten encapsular lógica, evitar repetición y construir programas más mantenibles.
- `def` define una función. La función solo se ejecuta cuando se llama. Los parámetros reciben argumentos y `return` devuelve resultados.
- Python permite valores por defecto, llamadas con nombre, devolución múltiple, `*args`, `**kwargs`, anotaciones de tipo y docstrings.
- El siguiente tema se centrará en clases y objetos (Programación orientada a Objetos).

```
# Siguiendo el paso
# Agrupar datos y comportamiento
# mediante clases y objetos

def resumen_final():
    return "Funciones preparadas para reutilizar código"

print(resumen_final())
```

Flujo de trabajo en el laboratorio

- Desde el repositorio Git podrás cargar el directorio del Tema03.

El laboratorio debe reforzar la creación de funciones con `def` para evitar la repetición de código, asegurando el uso correcto de parámetros, argumentos y `return` para devolver resultados, además de integrar anotaciones de tipo y docstrings.

- La versión de Python usada en el venv del curso será Python 3.13.

```
# Cargar el entorno
cd ~/Curso_Python
source .venv/bin/activate
python -version
# Actualizar del repositorio Git
git pull origin main
# Para ejecutar VSCode
code . # usar carpeta Tema03
# Para ejecutar en Terminal
python
```